

Unidad 5. Programación Orientada a Objetos (POO)

1. Introducción: Pensando en Objetos

La **Programación Orientada a Objetos (POO)** no es solo una sintaxis nueva; es un **paradigma de programación**. Un paradigma es un modelo o estilo fundamental de desarrollo de software.

Hasta ahora, has programado bajo el **paradigma estructurado** (o procedimental). En este enfoque, los datos (variables) y las acciones (funciones) están separados. El programador es como un cocinero que sigue una receta paso a paso: "coge estos huevos, bátelos, añade sal". Si el programa crece, terminamos con miles de variables sueltas y funciones que las modifican sin control, generando un código difícil de mantener ("código espagueti").

La POO propone un cambio radical: **modelar el software basándonos en las cosas (objetos) que existen en el problema real**. En lugar de separar datos y lógica, los empaquetamos juntos en unidades llamadas **Objetos**.



Documentación oficial

Java pertenece a la conocida empresa **Oracle**. Puedes consultar la [documentación oficial de Oracle](#) para ampliar tu conocimiento sobre **Programación Orientada a Objetos**.

1.1. Conceptos Fundamentales: La Tríada del Objeto

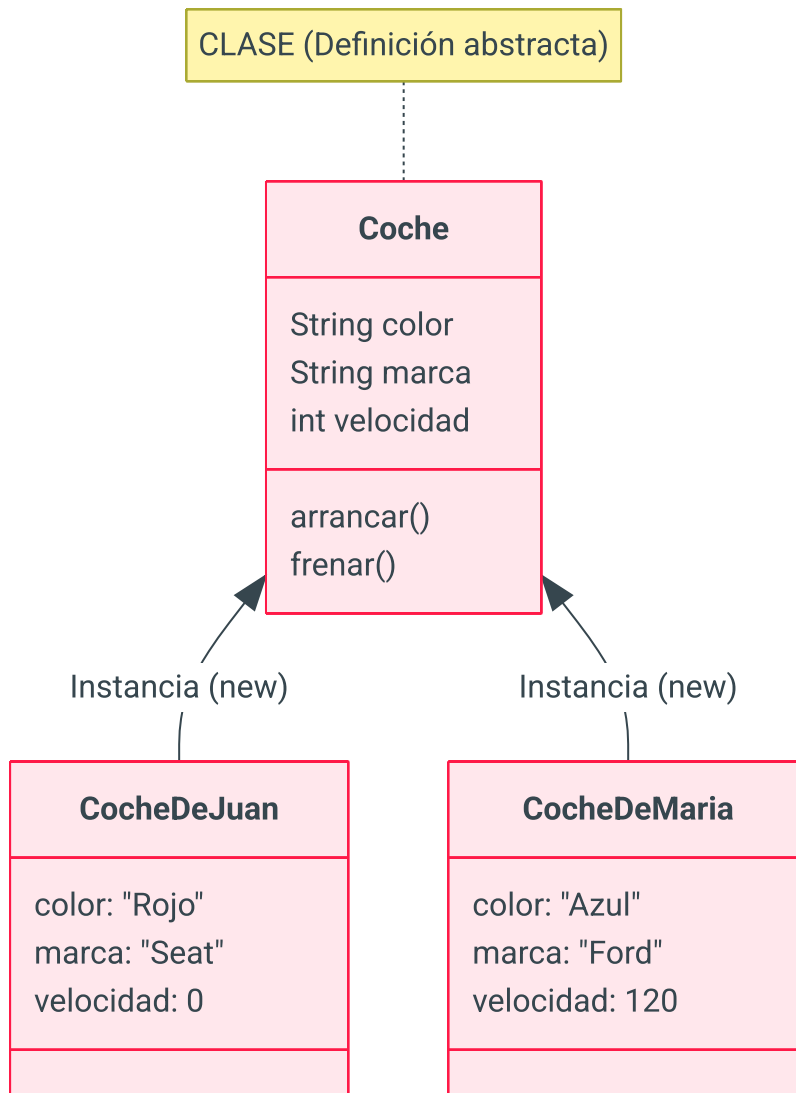
Para que algo sea considerado un objeto en este paradigma, debe cumplir con tres características esenciales que le dan sentido y autonomía:

1. **Identidad:** Es la propiedad que permite distinguir un objeto de otro, incluso si sus contenidos son idénticos. En la vida real, dos monedas de un euro son iguales, pero son dos objetos distintos (tienen identidad propia). En Java, esta identidad la gestiona la referencia de memoria.
2. **Estado (Atributos):** Son los datos que caracterizan al objeto en un momento dado. Representan las propiedades o características (ej: color, tamaño, velocidad actual).
3. **Comportamiento (Métodos):** Son las acciones que el objeto puede realizar o las respuestas que puede dar ante mensajes de otros objetos (ej: frenar, cambiar de color, calcular total).

1.2. Clase vs. Objeto: La Abstracción

Es vital distinguir entre el concepto abstracto y la realidad concreta.

- **La Clase (El plano):** Es la definición abstracta, el plano o la plantilla. Define qué atributos y métodos tendrán los objetos, pero **no contiene datos concretos** ni ocupa memoria de ejecución para almacenarlos.
- **El Objeto (La Instancia):** Es la concreción de la clase. Es un elemento tangible en la memoria del ordenador que se ha creado a partir de la clase.



1.3. Ejemplos de Abstracción en la Vida Real

El proceso de diseñar clases se llama **abstracción**: consiste en observar un objeto real, ignorar los detalles irrelevantes y quedarse solo con las características importantes para nuestro programa.

Veamos cómo modelaríamos diferentes objetos cotidianos en clases de Java:



A. Una Lámpara

Si queremos programar un sistema de domótica, no nos importa de qué material es la bombilla, solo si está encendida o no.

- **Estado (Atributos):** `estaEncendida` (boolean), `intensidad` (int).
- **Comportamiento (Métodos):** `encender()`, `apagar()`, `regularIntensidad(int nivel)`.

B. Una Cuenta Bancaria

Para un banco, tu cuenta es un objeto digital.

- **Estado:** `numeroCuenta` (String), `saldo` (double), `titular` (String).
- **Comportamiento:** `ingresar(double cantidad)`, `retirar(double cantidad)`, `consultarSaldo()`.

C. Un Enemigo de Videojuego

Para un juego de rol.

- **Estado:** `puntosVida` (int), `fuerzaAtaque` (int), `tipoArma` (String).
- **Comportamiento:** `atacar(Personaje objetivo)`, `recibirDano(int cantidad)`, `gritar()`.

2. Clases y Objetos: El Plano y la Casa

Para entender la diferencia fundamental entre estos dos conceptos, usaremos una analogía arquitectónica:

- **La Clase es el Plano (Blueprints):**
 - El arquitecto dibuja un plano detallado. En él especifica dónde van las ventanas, de qué material son las paredes y cuántas habitaciones hay.
 - Pero **nadie puede vivir en un plano**. El plano es solo papel (o un archivo digital). No tiene paredes físicas, ni techo, ni ocupa un lugar en la calle. Es pura información.
- **El Objeto es la Casa Construida:**
 - Usando ese **mismo plano**, una constructora puede edificar 50 casas en una urbanización.
 - Cada casa es una **instancia** del plano.
 - Aunque todas se hicieron con el mismo plano, cada casa es independiente:
 - En la Casa Nº 1 pueden pintar las paredes de azul.



- En la Casa Nº 2 pueden tener las luces encendidas.
- Si se rompe una ventana en la Casa Nº 3, no se rompe en la Casa Nº 4.

En programación:

- **La Clase** define la estructura.
- **El Objeto** ocupa memoria y tiene sus propios valores.

2.1. Definición de una Clase en Java

Una clase se define con la palabra reservada `class`. Por convención, **los nombres de las clases siempre empiezan por Mayúscula**.

Vamos a crear nuestro hilo conductor: un **Personaje de Videojuego**.



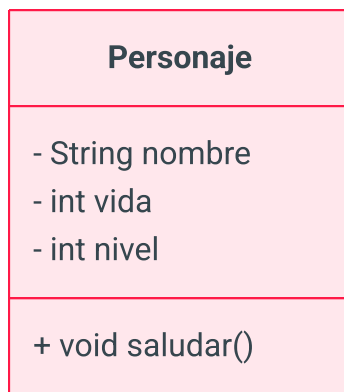
Comprobación

Comprueba el funcionamiento del programa. **Copia el código y ejecútalo** en tu entorno para comprender cómo funciona.

```
1 // Definición de la Clase (El Molde)
2 public class Personaje {
3
4     // ATRIBUTOS (Estado - Datos)
5     String nombre;
6     int vida;
7     int nivel;
8
9     // MÉTODOS (Comportamiento - Acciones)
10    void saludar() {
11        System.out.println("¡Hola! Soy " + nombre + " y tengo nivel " +
12        nivel);
13    }
14 }
```

Representación Visual (Diagrama de Clases UML):

Así es como representamos esta clase en un diagrama profesional:



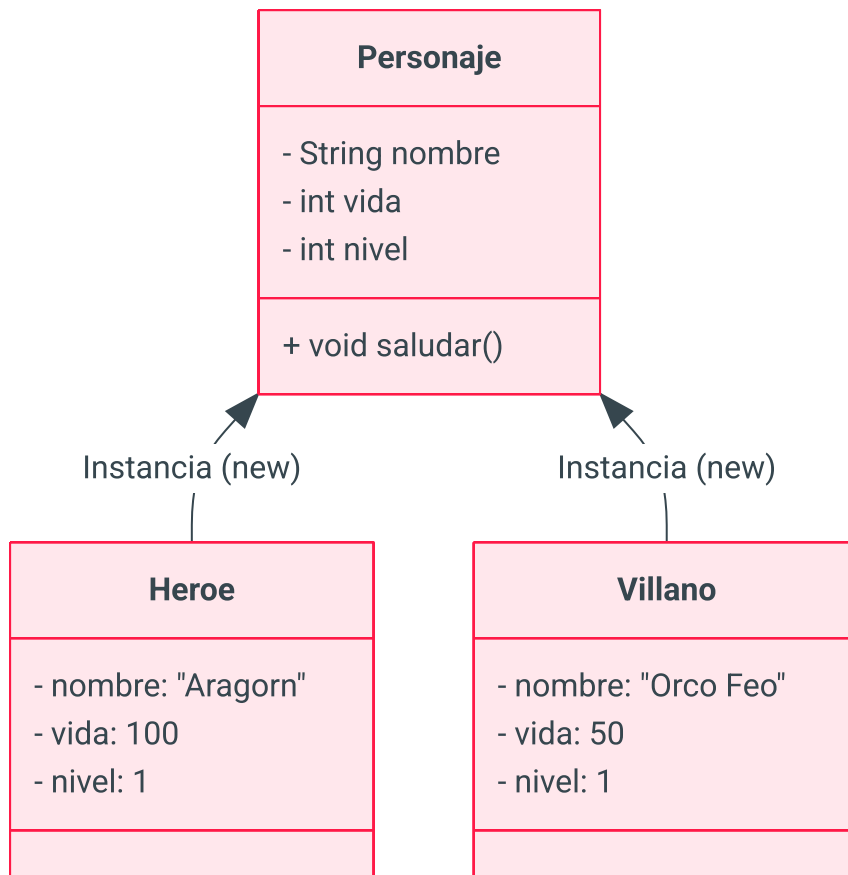
2.2. Instanciación: Creando Objetos

Para crear un objeto (una "casa") a partir de la clase, usamos el operador `new`.

```
1  public class Main {  
2      public static void main(String[] args) {  
3          // Declaración y creación del Objeto 1  
4          Personaje heroe = new Personaje();  
5  
6          // Accediendo a sus datos (¡Ojo! Aún no tienen valor, están  
7  vacíos)  
8          heroe.nombre = "Aragorn";  
9          heroe.vida = 100;  
10         heroe.nivel = 1;  
11  
12         // Declaración y creación del Objeto 2  
13         Personaje villano = new Personaje();  
14         villano.nombre = "Orco Feo";  
15         villano.vida = 50;  
16         villano.nivel = 1;  
17  
18         // Usando sus comportamientos  
19         heroe.saludar(); // Imprime: ¡Hola! Soy Aragorn...  
20         villano.saludar(); // Imprime: ¡Hola! Soy Orco Feo...  
21     }  
}
```

Diagrama de Instancias:

Aquí podemos ver cómo, a partir de una única clase `Personaje`, hemos creado dos objetos independientes en la memoria, cada uno con sus propios valores.



Momento de Práctica: Tu primer Objeto

1. Crea una clase llamada `Coche` en un archivo `Coche.java`.
2. Añádele atributos: `marca` (String), `modelo` (String) y `velocidad` (int).
3. Crea un método `void infoCoche()` que muestre un mensaje con la información del modelo y la marca.
4. En el `main`, crea dos coches distintos (ej: un Ferrari y un Seat, con sus modelos) y muestra su información.

3. Miembros de una Clase

Una clase se compone de dos tipos de miembros: Atributos y Métodos.

3.1. Atributos (Variables de Instancia)

Son las variables que guardan el estado de **cada objeto individualmente**. Cada objeto tiene su propia copia de estas variables.



- Si cambias la vida del `heroe`, la vida del `villano` NO cambia. Son independientes.

3.2. Métodos

Son las funciones que definen qué puede hacer el objeto. Pueden leer y modificar los atributos del propio objeto.

Vamos a ampliar nuestro **Personaje** con una nueva habilidad: `recibirDano(int puntos)`.

```
1 // Así queda la clase Personaje ampliada
2 public class Personaje {
3     String nombre;
4     int vida;
5     int nivel;
6
7     void saludar() {
8         System.out.println("¡Hola! Soy " + nombre + " y tengo nivel " +
9 nivel);
10    }
11
12    // NUEVO MÉTODO: Lógica de combate
13    void recibirDano(int puntos) {
14        // Modificamos el atributo 'vida' de ESTE objeto
15        vida = vida - puntos;
16
17        // Comprobamos el estado tras el cambio
18        if (vida <= 0) {
19            vida = 0; // No dejamos que sea negativa
20            System.out.println("[MUERTE] " + nombre + " ha sido
21 derrotado.");
22        } else {
23            System.out.println("[DAÑO] " + nombre + " ha recibido " +
24 puntos + " de daño. Vida restante: " + vida);
25        }
26    }
27 }
```



Momento de Práctica: ¡A la batalla!

Ahora te toca a ti probar este nuevo método. Vuelve a tu clase principal (`JuegoMain`) y modifica el código para simular una pelea:

1. Copia el código nuevo del método `recibirDano` dentro de tu clase `Personaje`.
2. En el `main`, haz que el `villano` reciba 30 puntos de daño.
3. Luego, haz que reciba otros 50 puntos.
4. ¿Qué pasa si recibe un golpe fatal de 100 puntos? Comprueba que sale el mensaje de derrota.

3.3. El Puntero `this` : ¿Quién soy yo?

En Java, cuando escribes código dentro de una clase, a veces necesitas referirte al propio objeto que está ejecutando el código, es decir, "MI atributo" o "MI método". Para eso usamos la palabra clave `this`.

- `this` es una referencia al **objeto actual** que está ejecutando el código.

Es muy útil para evitar ambigüedades cuando un parámetro se llama igual que un atributo. Esto se llama **Shadowing** (Sombreado).

```
1 // Forma INCORRECTA (Ambigüedad)
2 void cambiarNombre(String nombre) {
3     nombre = nombre; // ¿Cuál es cuál? Java asigna el parámetro a sí
4     mismo. ¡No hace nada!
5 }
6
7 // Forma CORRECTA (Usando this)
8 void cambiarNombre(String nombre) {
9     this.nombre = nombre; // "Mi atributo nombre" = "El parámetro nombre"
10 }
```

Aplicándolo a nuestro Personaje

Vamos a añadir un método para cambiar el nombre del personaje, usando `this` para diferenciar el atributo del parámetro.

```
1 public class Personaje {
2     String nombre;
3     int vida;
4     int nivel;
5
6     // ... otros métodos ...
7
8     // Método para renombrar al personaje
9     void cambiarNombre(String nombre) {
10         // Usamos 'this' para dejar claro que queremos modificar el
11         atributo de la clase
12         this.nombre = nombre;
13     }
14 }
```



Momento de Práctica: Ampliando el Coche

Vamos a mejorar tu clase `Coche`.

1. Añade un método `void frenar(int cantidad)` que reduzca la velocidad en la *cantidad* correspondiente.
2. Añade un método `void acelerar(int cantidad)` que acelere la velocidad en la *cantidad* dada.
3. **Lógica:** El coche no puede tener velocidad negativa. Si al frenar baja de 0, la velocidad debe quedarse en 0 y mostrar un mensaje "¡Coche detenido!".
4. **Lógica:** El coche no puede sobrepasar la velocidad máxima (por ejemplo 200). Si al acelerar pasa de 200, la velocidad debe quedarse en 200 y mostrar un mensaje "¡Velocidad máxima alcanzada!".
5. Prueba a acelerar y frenar en el `main`.

4. Encapsulamiento y Visibilidad (La Caja Negra)

El **Encapsulamiento** es el principio fundamental de la POO que nos dice: "**Oculto los detalles internos y muestra solo lo que sea seguro usar**".

4.1 ¿Por qué necesitamos proteger nuestros datos?

Imagina que estás conduciendo tu coche. Tienes acceso al volante, los pedales y la palanca de cambios (Interfaz Pública). Pero el fabricante del coche ha protegido el motor bajo un capó cerrado.

¿Qué pasaría si pudieras tocar los pistones del motor directamente mientras conduces?

1. **Integridad:** Podrías romper el motor sin querer.
2. **Seguridad:** Podrías hacerte daño.
3. **Complejidad:** Tendrías que saber mecánica avanzada solo para ir a comprar pan.

En programación ocurre lo mismo. Si dejas que cualquiera modifique la `vida` de tu personaje directamente (`heroe.vida = -5000`), tu programa podría romperse o comportarse de forma absurda (¡personajes zombies con vida negativa!).

En Java, usamos los **Modificadores de Acceso** para controlar esto:

- **public** : Acceso total. Es la "puerta abierta" de la clase.
- **private** : Acceso restringido. Solo la propia clase puede ver y tocar estos miembros.



Regla de Oro: Por defecto, los **atributos** deben ser siempre **private**.

4.2 Getters y Setters

Si hacemos los atributos privados, ¿cómo trabajamos con ellos? Usando métodos públicos controlados, conocidos como **Accessors**.

1. El Setter (Método de Modificación)

Se usa para **asignar** un valor a un atributo privado.

- **Ventaja:** Nos permite validar el dato antes de guardarlo. Si el dato es incorrecto, podemos rechazarlo o corregirlo.
- *Analogía:* Es como el guardia de seguridad de una discoteca. No deja entrar a cualquiera.

2. El Getter (Método de Lectura)

Se usa para **recuperar** el valor de un atributo privado.

- **Ventaja:** Nos permite controlar cómo se muestra el dato o incluso devolver un dato calculado.
- *Analogía:* Es como mirar a través de un cristal blindado. Puedes ver lo que hay, pero no tocarlo.

4.3. Métodos Ocultos (Privados)

A veces, una clase necesita realizar tareas internas complejas que no deben ser vistas ni usadas desde fuera. A estos métodos se les llama **Helpers** o métodos auxiliares, y se declaran como **private**.

Imagina un método que calcula si un golpe recibido es crítico. Solo la clase `Personaje` debe usarlo cuando recibe daño; nadie desde fuera debería poder llamar a `esGolpeCritico()` directamente.

```
1 private boolean esGolpeCritico() {  
2     // Lógica interna compleja (ej: números aleatorios)  
3     return Math.random() > 0.95; // 5% de probabilidad  
4 }
```

Si el `Personaje` recibe un *golpe crítico* (tiene un 5% de posibilidades de recibirlo), al daño recibido se le añade 10 puntos. Nuestro método `recibirDano` puede llamar **internamente** al nuevo método:

```
1 public void recibirDano(int puntos) {  
2  
3     this.vida -= puntos;
```

```

4
5         if(esGolpeCritico()) {
6             this.vida = this.vida - 10;
7             System.out.println("[GOLPE CRÍTICO] Recibe 10 ptos. extra de
8 daño");
9         }
10
11         if (this.vida <= 0) {
12             this.vida = 0;
13             System.out.println("[GAME OVER] " + this.nombre + " ha
14 muerto.");
15         } else {
16             System.out.println("[DAÑO] " + this.nombre + " tiene " +
this.vida + " ptos. de vida");
        }
    }
}

```

4.4. Código Final de la Clase Personaje

Aquí tienes la versión profesional de nuestra clase `Personaje`, aplicando encapsulamiento, constructores y métodos.



Comprobación

Comprueba el funcionamiento del programa. **Copia el código y ejecútalo** en tu entorno para comprender cómo funciona.

```

1  public class Personaje {
2
3      // 1. ATRIBUTOS (Privados para protegerlos)
4      private String nombre;
5      private int vida;
6      private int nivel;
7
8      // 2. MÉTODOS DE COMPORTAMIENTO
9      public void saludar() {
10         System.out.println("¡Hola! Soy " + this.nombre + " (Nv. " +
11 this.nivel + ")");
12     }
13
14     public void recibirDano(int puntos) {
15
16         this.vida -= puntos;
17
18         if(esGolpeCritico()) {
19             this.vida = this.vida - 10;
20             System.out.println("[GOLPE CRÍTICO] Recibe 10 ptos. extra de
21 daño");
22         }
23
24         if (this.vida <= 0) {

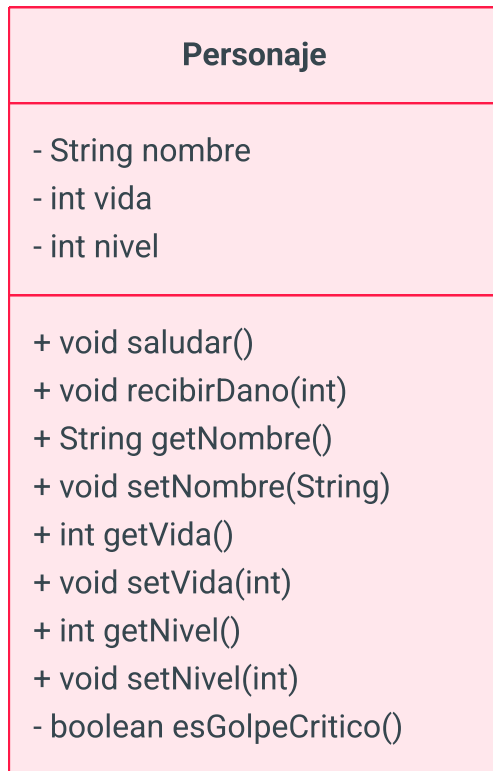
```

```
25         this.vida = 0;
26         System.out.println("[GAME OVER] " + this.nombre + " ha
27 muerto.");
28     } else {
29         System.out.println("[DAÑO] " + this.nombre + " tiene " +
30 this.vida + " ptos. de vida");
31     }
32 }
33
34 // 3. HELPERS. Métodos privados
35
36 private boolean esGolpeCritico() {
37     // Lógica interna compleja (ej: números aleatorios)
38     return Math.random() > 0.95; // 5% de probabilidad
39 }
40
41 // 4. GETTERS Y SETTERS (La Interfaz Pública)
42
43 // Getter para el nombre
44 public String getNombre() {
45     return this.nombre;
46 }
47
48 // Setter para el nombre (podríamos validar que no sea vacío)
49 public void setNombre(String nombre) {
50     this.nombre = nombre;
51 }
52
53 // Getter para la vida
54 public int getVida() {
55     return this.vida;
56 }
57
58 // Setter INTELIGENTE para la vida
59 public void setVida(int vida) {
60     if (vida < 0) {
61         System.out.println("Error: No se puede asignar vida negativa
62 a " + this.nombre);
63         this.vida = 0; // Corrección automática
64     } else {
65         this.vida = vida;
66     }
67 }
68
69 // Getter para el nivel
70 public int getNivel() {
71     return this.nivel;
72 }
73
74 // Setter para el nivel
75 public void setNivel(int nivel) {
76     this.nivel = nivel;
77 }
78 }
```

4.5. Diagrama de Clases UML (Encapsulado)

Observa cómo cambian los símbolos:

- - significa **private** (candado cerrado).
- + significa **public** (puerta abierta).



4.6. Ejemplo de Uso en el Main

Veamos cómo usamos esta clase segura. Fíjate que ya no podemos hacer `p.vida = -100`, el compilador no nos dejará.

```
1 public class Main {
2     public static void main(String[] args) {
3         System.out.println("--- INICIO DEL JUEGO ---");
4
5         // 1. Instanciación
6         Personaje heroe = new Personaje();
7
8         heroe.setNombre("Aragorn");
9         heroe.setVida(100);
10        heroe.setNivel(1);
11
12        Personaje villano = new Personaje();
13
14        villano.setNombre("Orco feo");
15        villano.setVida(50);
16        villano.setNivel(1);
```

```
17
18 // 2. Uso de Getters para leer información
19 System.out.println("El héroe se llama: " + heroe.getNombre());
20 System.out.println("Vida actual: " + heroe.getVida());
21
22 // 3. Intento de sabotaje (Asignar vida negativa)
23 // heroe.vida = -500; // ¡ERROR DE COMPILACIÓN! No es visible.
24
25 System.out.println("\n--- INTENTO DE TRAMPA ---");
26 heroe.setVida(-999); // Usamos el setter
27 System.out.println("Vida tras el intento: " + heroe.getVida());
28 // Sigue siendo 0, no -999
29
30 // 4. Combate legal
31 System.out.println("\n--- COMBATE ---");
32 villano.recibirDano(60); // El método gestiona la lógica interna
33 }
```

? Momento de Práctica: Protegiendo tu Coche

Vuelve a tu clase `Coche`.

1. Cambia la visibilidad de todos los atributos de la clase a `private`.
2. Intenta acceder directamente desde el main (`miCoche.velocidad = 200`). Verás que da error.
3. Crea un método `setVelocidad(int v)` que impida poner una velocidad negativa.
4. Usa el setter en el main para probarlo.

5. Constructores: Naciendo con Estado

Hasta este momento, hemos creado nuestros objetos en dos pasos:

1. Los creábamos vacíos con `new Personaje()`.
2. Les dábamos valores línea a línea usando los `setters`.

¿El problema? Es un proceso propenso a errores humanos. ¿Qué pasa si creas un `Personaje` pero se te olvida ponerle nombre o vida? Tendrías un objeto "zombie" o incompleto circulando por tu programa, lo cual podría causar errores graves más adelante.

El **Constructor** es un método especial diseñado para solucionar esto. Se ejecuta **automáticamente** en el momento exacto de la creación del objeto (al usar `new`). Su misión es garantizar que el objeto nazca en un estado válido.

5.1. Características Técnicas

Para que un método sea un constructor, debe cumplir dos reglas estrictas:

1. Tener **exactamente el mismo nombre** que la clase.
2. **No tener tipo de retorno** (ni siquiera `void`). No devuelve nada, devuelve el objeto mismo.

5.2. Tipos de Constructores

A. El Constructor por Defecto (El Invisible)

Si tú no escribes *ningún* constructor en tu clase, Java te regala uno invisible y vacío: `public Personaje() {}`. Este es el que nos permitía hacer `new Personaje()` hasta ahora.



Importante

En el momento en que escribas TÚ un constructor (con parámetros), Java retira el constructor por defecto. Si quieres seguir usándolo, tendrás que escribirlo explícitamente.

B. Constructores con Parámetros

Son los más útiles. Nos obligan a pasar los datos necesarios para que el objeto exista.

```
1  public class Personaje {
2      private String nombre;
3      private int vida;
4      private int nivel;
5
6      // Constructor: Obliga a dar un nombre, una vida y un nivel al nacer
7      public Personaje(String nombre, int vida, int nivel) {
8          this.nombre = nombre;
9          this.setVida(vida); // ¡Buena práctica! Usar el setter para
10         validar incluso al nacer
11          this.nivel = nivel;
12     }
13 }
```

Ahora, es **imposible** crear un personaje vacío. El compilador nos obligará a hacer: `new Personaje("Gandalf", 150, 5)`.

C. El Constructor de Copia

A veces necesitamos crear un objeto que sea una **copia exacta** de otro objeto que ya existe (un clon). Para eso usamos un constructor que recibe como parámetro un objeto de su misma clase.



```
1 // Constructor de Copia
2 public Personaje(Personaje p) {
3     this.nombre = p.nombre;
4     this.vida = p.vida;
5     this.nivel = p.nivel;
6 }
```

5.3. Sobrecarga de Constructores (Overloading)

A veces queremos flexibilidad. Quizás queremos crear un personaje dando todos los datos, o quizás solo dando el nombre y asumiendo valores por defecto para el resto.

Podemos tener múltiples constructores siempre que sus **parámetros sean diferentes** (en cantidad o tipo).

5.4. Resultado Final: La Clase Personaje Profesional

Aquí tienes cómo queda nuestra clase integrando **Encapsulamiento** (atributos privados y getters/setters) y **Constructores**.



Comprobación

Comprueba el funcionamiento del programa. **Copia el código y ejecútalo** en tu entorno para comprender cómo funciona.

```
1 public class Personaje {
2
3     // 1. ESTADO (Privado)
4     private String nombre;
5     private int vida;
6     private int nivel;
7
8     // 2. CONSTRUCTORES (Sobrecarga)
9
10    // Opción A: Constructor completo (Control total)
11    public Personaje(String nombre, int vida, int nivel) {
12        this.nombre = nombre;
13        this.setVida(vida); // Usamos setter para validar
14        this.setNivel(nivel);
15    }
16
17    // Opción B: Constructor rápido (Valores por defecto)
18    // Solo pide el nombre. Asume vida=100 y nivel=1.
19    public Personaje(String nombre) {
20        this.nombre = nombre;
21        this.vida = 100;
22        this.nivel = 1;
23    }
24 }
```



```
25 // Opción C: Constructor copia
26 // Recibe otro personaje y copia sus atributos
27 public Personaje(Personaje p) {
28     this.nombre = p.nombre;
29     this.vida = p.vida;
30     this.nivel = p.nivel;
31 }
32
33 // 3. MÉTODOS (Comportamiento)
34 public void saludar() {
35     System.out.println("¡Hola! Soy " + this.nombre + " (Nv. " +
36 this.nivel + ")");
37 }
38
39 public void recibirDano(int puntos) {
40
41     this.vida -= puntos;
42
43     if(esGolpeCritico()) {
44         this.vida = this.vida - 10;
45         System.out.println("[GOLPE CRÍTICO] Recibe 10 ptos. extra de
46 daño");
47     }
48
49     if (this.vida <= 0) {
50         this.vida = 0;
51         System.out.println("[GAME OVER] " + this.nombre + " ha
52 muerto.");
53     } else {
54         System.out.println("[DAÑO] " + this.nombre + " tiene " +
55 this.vida + " ptos. de vida");
56     }
57 }
58
59 // 4. HELPERS. Métodos privados
60
61 private boolean esGolpeCritico() {
62     // Lógica interna compleja (ej: números aleatorios)
63     return Math.random() > 0.95; // 5% de probabilidad
64 }
65
66 // 5. GETTERS Y SETTERS
67 public String getNombre() { return nombre; }
68 public void setNombre(String nombre) { this.nombre = nombre; }
69
70 public int getVida() { return vida; }
71 public void setVida(int vida) {
72     if (vida < 0) {
73         System.out.println("Error: Vida negativa no permitida.");
74         this.vida = 0;
75     } else {
76         this.vida = vida;
77     }
78 }
79
80 public int getNivel() { return nivel; }
81 public void setNivel(int nivel) {
```



```

        if (nivel < 1) nivel = 1; // Mínimo nivel 1
        else this.nivel = nivel;
    }
}

```

5.5. Uso en el Programa Principal

Fíjate cómo la creación de objetos ahora es mucho más robusta y limpia.

```

1  public class Main {
2      public static void main(String[] args) {
3
4          // Usando el Constructor Completo
5          Personaje heroe = new Personaje("Aragorn", 150, 10);
6
7          // Usando el Constructor Rápido
8          Personaje villano = new Personaje("Goblin"); // Vida 100, Nivel 1
9          automáticos
10
11         heroe.saludar();
12         villano.saludar();
13
14         // Intento de crear un personaje vacío... ¡ERROR DE COMPILACIÓN!
15         // Personaje fantasma = new Personaje(); // Java te dice: "¿Dónde
16         están los datos?"
17
18         // Personaje copia "clonamos" el villano
19
20         Personaje villano2 = new Personaje(villano); // villano2 es una
           copia idéntica de villano
       }
   }

```



Momento de Práctica: La Fábrica de Coches

Vamos a profesionalizar tu clase `Coche`.

1. Elimina cualquier inicialización manual que tuvieras en el `main`.
2. Crea un **Constructor Completo** que pida `marca`, `modelo` y `velocidad` (los datos los pedirá al usuario antes de llamar al constructor).
3. Crea un **Constructor Parcial** que solo pida `marca` y `modelo` (la velocidad debe iniciarse a 0).
4. Crea un **Constructor Copia**.
5. En el `main`, intenta crear un coche con `new Coche()`. ¿Qué ocurre?
6. Crea dos coches usando los dos constructores nuevos que has programado y muestra sus datos.
7. Copia en un tercer coche, el coche segundo.



6. Miembros Estáticos (`static`)

Hasta ahora, cada objeto tenía su propia copia de los atributos (cada `Personaje` tenía su propio `nombre`). Pero, ¿y si queremos un dato que sea **compartido** por todos?

Imagina un contador de **"Total de Personajes Creados"**.

- Si se lo damos a "Aragorn", solo él sabría el dato.
- Si se lo damos a "Legolas", tendríamos el dato duplicado.
- Ese dato no pertenece a un personaje concreto, pertenece al **JUEGO** (a la Clase).

Para eso usamos la palabra clave `static`.

- **Atributo Estático:** Existe una única copia en memoria para toda la aplicación. Si un objeto lo modifica, cambia para todos.
- **Método Estático:** Se puede llamar sin crear ningún objeto (como `Math.sqrt()`). No puede usar `this` porque no está ligado a ninguna instancia concreta.

```
1 public class Personaje {
2     // Variable de instancia (cada uno tiene la suya)
3     private String nombre;
4
5     // Variable de clase (COMPARTIDA por todos)
6     private static int contadorPersonajes = 0;
7
8     public Personaje(String nombre) {
9         this.nombre = nombre;
10        // Cada vez que nace uno, aumentamos el contador global
11        Personaje.contadorPersonajes++;
12    }
13 }
```

Podríamos encapsular el atributo `contadorPersonajes` para que solo pueda ser modificado por el constructor. Si además, necesitamos acceder a su valor, podemos crear un **Getter**.

```
1 public static int getTotalPersonajes() {
2     return contadorPersonajes;
3 }
```

Uso:

```
1 Personaje p1 = new Personaje("A");
2 Personaje p2 = new Personaje("B");
3
4 // Accedemos a través de la CLASE, no del objeto
5
```



```
System.out.println("Total creados: " + Personaje.getTotalPersonajes()); //  
Imprime 2
```

6.1 Constantes estáticas

En la UD1 estudiamos el uso de constantes, que son aquellas que se identifican con la palabra `final`.

Este concepto está muy relacionado con los atributos estáticos. Por ejemplo el método `Math.PI` tiene la siguiente forma, donde puedes observar el uso de la palabra `static`:

```
1 public static final double PI = 3.14159265358979323846;
```

Esto nos permite utilizar el valor de PI desde cualquier programa, sin necesidad de instanciar un objeto de la clase `Math`. Excepcionalmente este tipo de atributos se crean públicos, por lo que no incluye `getter`.

Nuestra Clase Personaje también puede mejorarse utilizando esta técnica. Vamos a imaginar que la vida de nuestro personaje está limitada a 500. Para ello definimos la siguiente constante estática:

```
1 public static final int VIDA_MAX = 500;
```

Esto nos permitiría hacer mejoras a nuestro código del siguiente tipo:

```
1 public void setVida(int vida) {  
2     if (vida < 0) {  
3         System.out.println("Error: Vida negativa no permitida.");  
4         this.vida = 0;  
5     } else if (vida > Personaje.VIDA_MAX){  
6         System.out.println("Error: Vida máxima excedida");  
7         this.vida = Personaje.VIDA_MAX;  
8     } else {  
9         this.vida = vida;  
10    }  
11 }
```

6.2. Código Final de la Clase Personaje

Aquí tienes la clase definitiva, integrando **Encapsulamiento, Constructores, Lógica y Miembros Estáticos**.



Comprobación

Comprueba el funcionamiento del programa. **Copia el código y ejecútalo** en tu entorno para comprender cómo funciona.

```
1  public class Personaje {
2
3      // --- 1. ATRIBUTOS ESTÁTICOS (De la Clase) ---
4      // Contador compartido por TODOS los objetos Personaje
5      private static int contadorPersonajes = 0;
6      // Vida máxima compartida por TODOS los objetos Personaje
7      public static final int VIDA_MAX = 500;
8
9      // --- 2. ATRIBUTOS DE INSTANCIA (Del Objeto) ---
10     private String nombre;
11     private int vida;
12     private int nivel;
13
14     // --- 3. CONSTRUCTORES ---
15     public Personaje(String nombre, int vida, int nivel) {
16         this.nombre = nombre;
17         this.setVida(vida); // Usamos setter para validar
18         this.setNivel(nivel); // Usamos setter para validar
19
20         // ¡IMPORTANTE! Cada vez que nace un personaje, aumentamos el
21         contador global
22         Personaje.contadorPersonajes++;
23     }
24
25     // Constructor rápido
26     public Personaje(String nombre) {
27         this(nombre, 100, 1); // Llamada al otro constructor
28     }
29
30     // Constructor copia
31     public Personaje(Personaje p) {
32         this(p.nombre, p.vida, p.nivel);
33     }
34
35     // --- 4. MÉTODOS DE COMPORTAMIENTO ---
36     public void saludar() {
37         System.out.println("¡Hola! Soy " + this.nombre + " (Nv. " +
38         this.nivel + ")");
39     }
40
41     public void recibirDano(int puntos) {
42
43         this.vida -= puntos;
44
45         if(esGolpeCritico()) {
46             this.vida = this.vida - 10;
47             System.out.println("[GOLPE CRÍTICO] Recibe 10 ptos. extra de
48         daño");
49     }
```

```
50
51     if (this.vida <= 0) {
52         this.vida = 0;
53         System.out.println("[GAME OVER] " + this.nombre + " ha
54 muerto.");
55     } else {
56         System.out.println("[DAÑO] " + this.nombre + " tiene " +
57 this.vida + " ptos. de vida");
58     }
59 }
60
61 private boolean esGolpeCritico() {
62     return Math.random() > 0.95;
63 }
64
65 // --- 5. MÉTODOS ESTÁTICOS ---
66 // Permite consultar el contador sin tener ningún personaje creado
67 public static int getTotalPersonajes() {
68     return contadorPersonajes;
69 }
70
71 // --- 6. GETTERS Y SETTERS ---
72 public String getNombre() { return nombre; }
73 public void setNombre(String nombre) { this.nombre = nombre; }
74
75 public int getVida() { return vida; }
76 public void setVida(int vida) {
77     if (vida < 0) {
78         System.out.println("Error: Vida negativa no permitida.");
79         this.vida = 0;
80     } else if (vida > Personaje.VIDA_MAX) {
81         System.out.println("Error: Vida máxima excedida");
82         this.vida = Personaje.VIDA_MAX;
83     } else {
84         this.vida = vida;
85     }
86 }
87
88 public int getNivel() { return nivel; }
89 public void setNivel(int nivel) {
90     if (nivel < 1) {
91         System.out.println("Error: Nivel inferior al mínimo (1).");
92         this.nivel = 1;
93     } else {
94         this.nivel = nivel;
95     }
96 }
97 }
```

Los miembros estáticos suelen representarse subrayados. En este diagrama, el contador es estático.



Personaje
- String nombre - int vida - int nivel - <u>static int contadorPersonajes</u> - <u>static final int VIDA_MAX</u>
+ Personaje(String, int, int) + Personaje(String) + Personaje(Personaje) + void saludar() + void recibirDano(int) + String getNombre() + void setNombre(String) + int getVida() + void setVida(int) + int getNivel() + void setNivel(int) - boolean esGolpeCritico() + <u>static int getTotalPersonajes()</u>

6.3. Ejemplo de Uso en el Main

Observa cómo accedemos a la información estática usando el **Nombre de la Clase** (`Personaje.algo`), no el nombre de un objeto.

```

1  public class JuegoMain {
2      public static void main(String[] args) {
3
4          System.out.println("--- INICIO DEL JUEGO ---");
5
6          // 1. Consultar estático antes de crear nada
7          System.out.println("Personajes actuales: " +
8  Personaje.getTotalPersonajes()); // Salida: 0
9
10         // 2. Crear objetos (El constructor aumentará el contador solo)
11         Personaje p1 = new Personaje("Aragorn", 150, 10);
12         Personaje p2 = new Personaje("Legolas", 120, 10);
13         Personaje p3 = new Personaje("Gimli");
14
15         // 3. Consultar estático de nuevo

```

```
16         System.out.println("Personajes actuales: " +
17         Personaje.getTotalPersonajes()); // Salida: 3
18
19         // 4. Probar encapsulamiento y métodos
20         p1.recibirDano(50);
21         p1.setVida(-200); // El setter lo corregirá a 0
22
23         System.out.println("Vida de Aragorn: " + p1.getVida());
    }
}
```

? Momento de Práctica: La Fábrica de Coches (Final)

Para terminar tu clase `Coche`, vamos a añadirle control de producción.

1. Añade un atributo privado y estático `contadorCoches`.
2. Modifica tus constructores para que aumenten este contador cada vez que se cree un coche nuevo.
3. Crea un método público y estático `getCochesFabricados()` que devuelva ese número.
4. Añade también una **Constante Estática** (`public static final`) llamada `RUEDAS = 4` (todos los coches tienen 4 ruedas).
5. En el `main`:
 - Imprime cuántas ruedas tiene un coche usando la constante.
 - Crea 3 coches.
 - Imprime el total de coches fabricados llamando al método estático.

Unidad 6. Programación Orientada a Objetos Avanzada

1. Clases Compuestas (Composición)

Hasta ahora, hemos trabajado con objetos sencillos (como un `Coche` con velocidad o color). Sin embargo, en el mundo profesional, el software se parece más a un juego de **LEGO**.

¿Cómo construyes un castillo de Lego? No fabricas el plástico desde cero. Tomas bloques que ya existen (ventanas, puertas, ladrillos) y los "compones" para crear algo nuevo y más complejo.

La **Composición** es exactamente eso: crear clases complejas usando otras clases como "piezas".

1.1. La Relación "TIENE-UN" (Has-A)

Para saber si estás ante un caso de composición, usa la prueba del "TIENE-UN":

- Un `Ordenador` **TIENE UN** `Procesador`.
- Un `Personaje` **TIENE UN** `Arma`.
- Una `Casa` **TIENE UNA** `Cocina`.

(Si la frase fuera "ES UN", estaríamos hablando de Herencia, que veremos luego).



Consejo Pro

En Java, implementamos esto declarando atributos que son **referencias a otros Objetos**, en lugar de simples `int` o `String`.

1.2. Ventajas: Delegación

La clase contenedora (`Personaje`) no necesita saber cómo dispara el `Arma`. Solo le pide que dispare. A esto se le llama **Delegación**: "Yo (Personaje) no calculo el daño, *delego* esa tarea a mi arma".

1.3. Ejemplo: Personaje y Arma

 Vamos a ver cómo se traduce esto a código real.

```
1 // PIEZA 1: El objeto "pequeño"
2 public class Arma {
3     private String nombre;
4     private int dano;
5
6     public Arma(String nombre, int dano) {
7         this.nombre = nombre;
8         this.dano = dano;
9     }
10
11     public int getDano() { return dano; }
12     public String getNombre() { return nombre; }
13 }
```

```
1 // PIEZA 2: El objeto "contenedor"
2 public class Personaje {
3     private String nombre;
4     private int vida;
5
6     // COMPOSICIÓN: El personaje TIENE un Arma
7     // En memoria, esto es una referencia (un hueco) para conectar un
8     objeto Arma.
9     private Arma armaEquipada;
10
11     public Personaje(String nombre, Arma armaInicial) {
12         this.nombre = nombre;
13         // Importante: Validamos que no nos pasen un arma 'null'
14         setArmaEquipada(armaInicial);
15     }
16
17     public void atacar() {
18         // DELEGACIÓN: Usamos el objeto interno para trabajar
19         System.out.println(this.nombre + " ataca con " +
20 this.armaEquipada.getNombre());
21         System.out.println("Daño causado: " +
22 this.armaEquipada.getDano());
23     }
24
25     public void cambiarArma(Arma nuevaArma) {
26         setArmaEquipada(nuevaArma);
27         System.out.println(this.nombre + " ahora lleva: " +
28 nuevaArma.getNombre());
29     }
30
31     public Arma getArmaEquipada() {
32         return armaEquipada;
33     }
34
35     public void setArmaEquipada(Arma nuevoArma) {
36         if (nuevoArma != null) {
37             this.armaEquipada = nuevoArma;
38         } else {
39             System.out.println(";Cuidado! Se intenta crear sin arma.");
40             Arma punyoPorDefecto = new Arma("Puño", 1);
41         }
42     }
43 }
```

```

40         this.armaEquipada = punyoPorDefecto;
41     }
    }
}

```

1.4. Composición con Colecciones (Relación "TIENE-MUCHOS")

¿Y si nuestro personaje tiene una mochila? Una mochila no tiene un solo objeto, tiene **muchos**. Aquí es donde combinamos Composición con `ArrayList`.

```

1  import java.util.ArrayList;
2
3  // CLASE COMPONENTE
4  public class Item {
5      private String nombre;
6      private int durabilidad; // Ejemplo: 100 (nuevo) a 0 (roto)
7
8      public Item(String nombre, int durabilidad) {
9          this.nombre = nombre;
10         this.durabilidad = durabilidad;
11     }
12
13     @Override
14     public String toString() { return nombre + " [" + durabilidad + "%]"; }
15 }

```

```

1  // CLASE COMPUESTA INTERMEDIA
2  public class Mochila {
3      private ArrayList<Item> items; // La colección
4      private int capacidadMax;
5
6      public Mochila(int capacidad) {
7          this.items = new ArrayList<>();
8          this.capacidadMax = capacidad;
9      }
10
11     public void guardar(Item i) {
12         if (items.size() < capacidadMax) {
13             items.add(i);
14             System.out.println("Guardado: " + i);
15         } else {
16             System.out.println("No cabe: " + i);
17         }
18     }
19
20     public String listar() {
21         return items.toString();
22     }
23 }

```



```
1 // CLASE PRINCIPAL
2 public class Personaje {
3
4     private String nombre;
5     private int vida;
6
7     private Arma armaEquipada;
8     private Mochila mochila;
9
10    public Personaje(String nombre, Arma armaInicial) {
11        this.nombre = nombre;
12        // Importante: Validamos que no nos pasen un arma 'null'
13        if (armaInicial == null) {
14            System.out.println(";Cuidado! Se intenta crear sin arma.");
15        }
16        setArmaEquipada(armaInicial);
17        mochila = new Mochila(10);
18    }
19
20
21    public String getNombre() {
22        return nombre;
23    }
24
25    public void setNombre(String nombre) {
26        this.nombre = nombre;
27    }
28
29    public int getVida() {
30        return vida;
31    }
32
33    public void setVida(int vida) {
34        this.vida = vida;
35    }
36
37    public Arma getArmaEquipada() {
38        return armaEquipada;
39    }
40
41    public void atacar() {
42        // DELEGACIÓN: Usamos el objeto interno para trabajar
43        System.out.println(this.nombre + " ataca con " +
44            this.armaEquipada.getNombre());
45        System.out.println("Daño causado: " +
46            this.armaEquipada.getDano());
47    }
48
49    public void cambiarArma(Arma nuevaArma) {
50        setArmaEquipada(nuevaArma);
51        System.out.println(this.nombre + " ahora lleva: " +
52            getArmaEquipada().getNombre());
53    }
54
55    public void setArmaEquipada(Arma nuevoArma) {
56        if (nuevoArma != null) {
57            this.armaEquipada = nuevoArma;
```

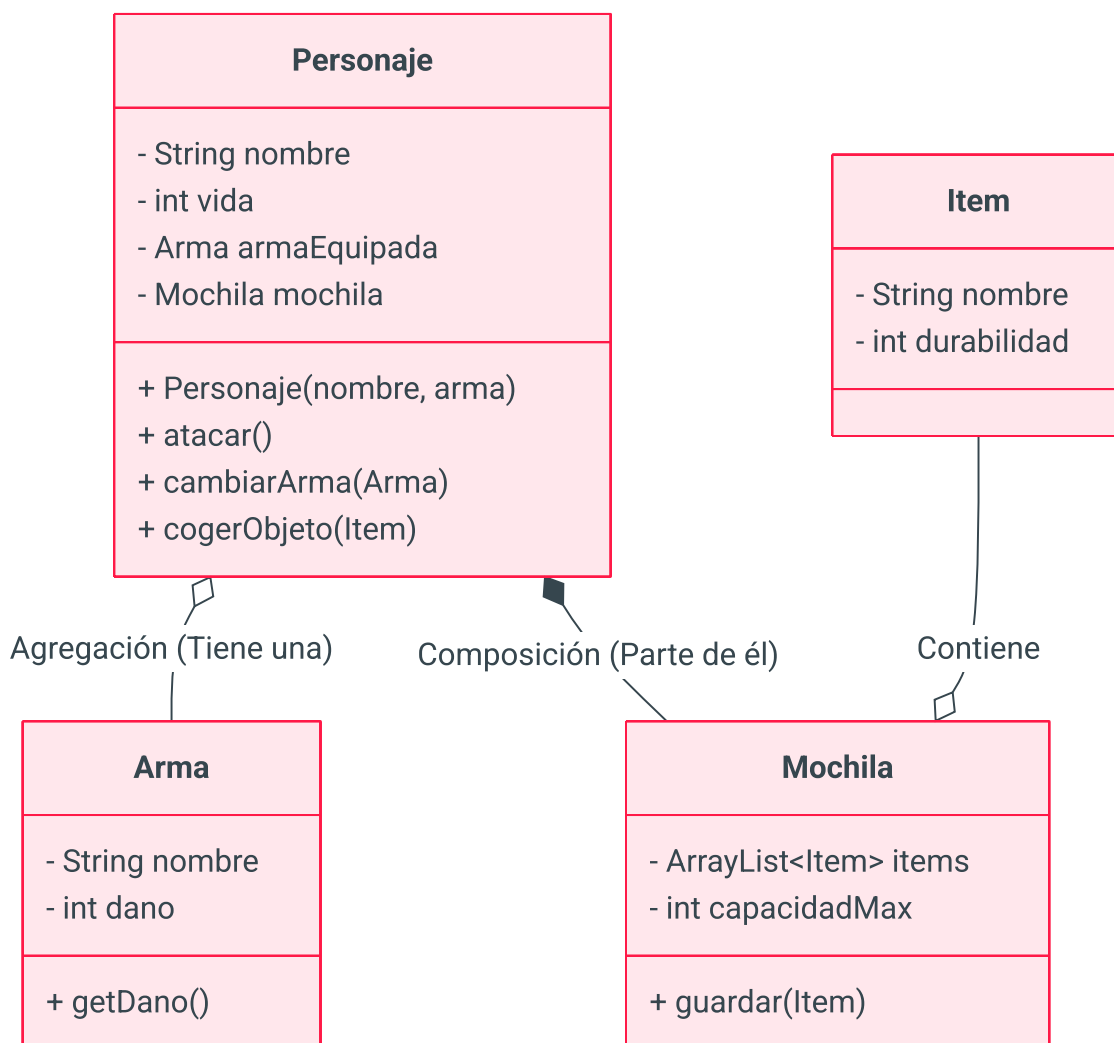
```

58         } else {
59             Arma punyoPorDefecto = new Arma("Puño", 1);
60             this.armaEquipada = punyoPorDefecto;
61         }
62     }
63
64     public void cogerObjeto(Item i) {
65         // Delegamos en la mochila la acción de guardar
        this.mochila.guardar(i);
    }
}

```

1.5. Visualización Completa

En este diagrama mostramos **todos** los atributos para ver la estructura completa de objetos en memoria.





Momento de Práctica: El Ordenador Modular

Vamos a construir un PC modular. Esta vez no es una sola clase, será un sistema de piezas ensambladas.

1. Las Piezas (Clases Componentes)

- **Clase Procesador :**
 - Atributos: `marca` (String) y `modelo` (String).
 - Constructor y método `toString()` que devuelva algo como *"Intel i7"*.
- **Clase MemoriaRAM :**
 - Atributos: `capacidad` (int, en GB) y `tecnologia` (String, ej: "DDR4", "DDR5").
 - Constructor y método `toString()`.

2. La Torre (Clase Compuesta)

- **Clase Ordenador :**
 - Atributos:
 - `cpu`: Objeto de tipo `Procesador`.
 - `bancosRAM`: `ArrayList<MemoriaRAM>` (Aquí guardaremos los módulos).
 - `MAX_RAM_SLOTS`: Constante `final int` (ej: 4).
 - Constructor: Debe recibir **solo el Procesador** y crear la lista de RAM vacía.

3. La Lógica de Ensamblaje

- Método `void anadirMemoria(MemoriaRAM ram) :`
 - Comprueba si hay hueco libre (`size() < MAX_RAM_SLOTS`).
 - Si hay hueco, añade la memoria a la lista.
 - Si no, imprime: *"¡No quedan slots libres!"*.
- Método `void mostrarConfiguracion() :`
 - Imprime el procesador.
 - Recorre la lista de RAM sumando sus capacidades.
 - Muestra el total: *"Ordenador con [Intel i7] y [32] GB de RAM instalada"*.

Reto Extra: Modifica `anadirMemoria` para que impida mezclar tecnologías. Si la primera RAM es "DDR4", no debe dejar añadir una "DDR5".

2. Introducción a la Herencia



La **Herencia** es el mecanismo que nos permite crear nuevas clases basadas en clases que ya existen.

Imagina que estás programando un videojuego con 50 tipos de enemigos distintos (Zombies, Aliens, Robots...). Todos tienen `vida`, todos se `mueven`, todos tienen `coordenadas`.

- ¿Vas a escribir el código de `moverse()` 50 veces? **NO**.
- ¿Si descubres un fallo en el movimiento, vas a corregirlo en 50 archivos? **NO**.

Aquí entra la **Herencia**: Igual que tú heredas el ADN de tus padres (no tienes que "aprender" a tener ojos, ya te vienen de fábrica), en programación creamos una **Clase Padre** con lo común y las **Clases Hijas** heredan su código automáticamente.

2.1. La Regla del "ES-UN" (Is-A)

Para saber si debes usar herencia, la frase debe tener sentido:

- Un `Guerrero` **ES UN** `Personaje`. (Correcto: Herencia)
- Un `Coche` **ES UN** `Motor`. (Incorrecto: Es Composición, porque *TIENE un* motor)

2.2. Sintaxis (`extends`) y Visibilidad (`protected`)

Para heredar usamos `extends`. Pero tenemos un problema: los hijos no pueden ver los atributos `private` de sus padres.

Para solucionarlo, existe un modificador de acceso diseñado específicamente para la herencia: `protected`. Este nivel de visibilidad actúa como un punto intermedio entre la privacidad total y la exposición pública:

Modificador	Alcance y Descripción
<code>public</code>	Visible desde cualquier clase de la aplicación.
<code>protected</code>	Visible para las clases del mismo paquete y para sus subclases (hijas), independientemente de donde estén.
<code>private</code>	Visible únicamente dentro de la propia clase (ni siquiera los hijos pueden acceder).



¿Cuándo usar `protected`?

Aunque `protected` es útil para la herencia, **no abuses de él**.

Por defecto, intenta mantener tus atributos `private` y usa **Getters/Setters** (públicos o protegidos) si los hijos necesitan acceder. Usa `protected` en atributos solo cuando sea estrictamente necesario para el diseño o rendimiento.

```
1 // CLASE PADRE (Superclase)
2 public class Personaje {
3     protected String nombre; // Protected para que los hijos lo puedan
4     usar
5     protected int vida;
6
7     public Personaje(String nombre, int vida) {
8         this.nombre = nombre;
9         this.vida = vida;
10    }
11
12    public void saludar() {
13        System.out.println("Hola, soy un personaje genérico.");
14    }
15 }
```

```
1 // CLASE HIJA (Subclase)
2 public class Guerrero extends Personaje {
3     private int furia; // Atributo exclusivo del hijo
4
5     // EL CONSTRUCTOR: La Regla de los Cimientos
6     public Guerrero(String nombre, int vida, int furia) {
7         // 1º Construimos al padre (Cimientos)
8         super(nombre, vida);
9         // 2º Construimos al hijo (Decoración)
10        this.furia = furia;
11    }
12 }
```



Importante: `super()`

La llamada a `super()` debe ser SIEMPRE la **primera línea** del constructor del hijo. No puedes decorar la casa si no has puesto los cimientos antes.



Curiosidad: ¿Herencia Múltiple?

En Java, una subclase **SOLO puede tener un PADRE** (`extends`). No existe la herencia múltiple de clases (como en C++ o Python).

Esto se hace para evitar conflictos (el "Problema del Diamante"). Sin embargo, Java compensa esto permitiendo implementar **múltiples Interfaces**.

2.3. Sobreescritura de Métodos (`@Override`)

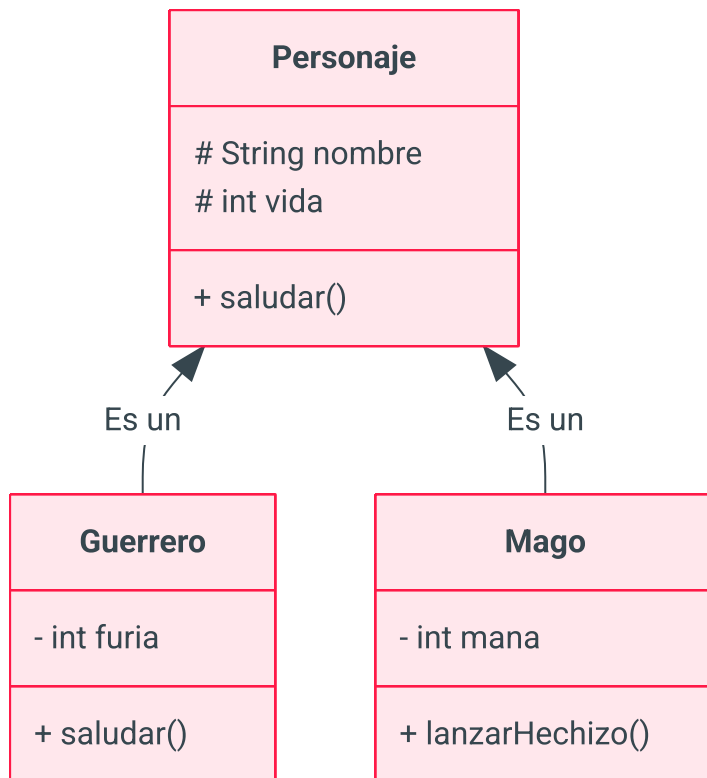
¿Y si el hijo quiere hacer las cosas de forma diferente a su padre? El `Guerrero` hereda `saludar()`, pero quiere saludar gritando.

Podemos **reescribir** el método usando la etiqueta `@Override`.

```
1  public class Guerrero extends Personaje {
2      // ... constructor ...
3
4      @Override
5      public void saludar() {
6          // Opción A: Borrón y cuenta nueva (Ignoramos al padre)
7          System.out.println("¡GRRR! SOY " + this.nombre + " Y TE
8  APLASTARÉ.");
9
10         // Opción B: Ampliar lo que hace el padre
11         // super.saludar(); // Llamaria al "Hola..." original
12         // System.out.println("...pero tengo mal genio.");
13     }
14 }
```

2.4. Diagrama de Herencia

En UML, la herencia se representa con una **flecha de línea continua y punta triangular hueca** que va del Hijo al Padre.



(Nota: El símbolo # representa *protected* en los diagramas).



Momento de Práctica: El Reino Animal

Vamos a crear una pequeña jerarquía biológica.

1. Crea una clase **Animal** con atributos *protected* nombre y edad .
 - Método `hacerSonido()` que imprima: `"..."` (sonido genérico).
2. Crea la clase **Perro** que hereda de **Animal** .
 - `@Override` `hacerSonido()` -> `"¡Guau!"`.
3. Crea la clase **Gato** que hereda de **Animal** .
 - `@Override` `hacerSonido()` -> `"¡Miau!"`.
4. En el Main:
 - Crea un Perro y un Gato.
 - Haz que ambos "hablen".
 - **Prueba Curiosa:** Intenta hacer `Animal a = new Perro(...)` . ¿Funciona? (Spoiler: Sí, es el principio del Polimorfismo).

3. Clases Abstractas

Hasta ahora, nuestras clases servían para crear objetos (`new Guerrero()`). Pero a veces, necesitamos definir un concepto que es puramente estructural y no debería existir por sí solo.

Concepto Abstracto vs Implementación Concreta: Piensa en la ingeniería. Tú sabes lo que es un **"Vehículo"** (tiene motor, ruedas, transporta gente).

- Pero tú no puedes ir a un concesionario y decir: *"Deme un Vehículo, a secas"*.
- El vendedor te dirá: *"¿Quiere un Coche? ¿Una Moto? ¿Un Camión?"*.

"Vehículo" es un concepto abstracto. No se puede fabricar. Solo sirve de base para crear cosas reales como "Coche" o "Moto".

3.1. Definición (`abstract`)

En Java, usamos la palabra clave `abstract` para impedir que una clase se pueda instanciar.

```
1 public abstract class Figura {
2     protected String color; // Un atributo común más lógico para una
3     figura
4
5     public Figura(String color) {
6         this.color = color;
7     }
8 }
```

- **Efecto inmediato:** `new Figura("Rojo")` dará **ERROR DE COMPILACIÓN**.
- **Utilidad:** Solo podemos usarla como padre (`extends`) para crear clases útiles como `Circulo` o `Rectangulo`.

3.2. Métodos Abstractos (El Contrato)

Si definimos una "Figura", sabemos que todas las figuras tienen un **Área**. Pero, ¿cómo se calcula el área de "una figura genérica"? ¡No se puede! Depende de si es redonda o cuadrada.

Aquí usamos un **Método Abstracto**: Definimos el **QUÉ** pero dejamos el **CÓMO** vacío.

```
1 public abstract class Figura {
2     protected String color;
3
4     public Figura(String color) {
5         this.color = color;
6     }
7
8     // MÉTODO ABSTRACTO: No tiene llaves {}. Termina en "punto y coma".
9     // "Prometo que cualquier hijo mío sabrá calcular su área, pero yo no
10    sé cómo".
11    public abstract double calcularArea();
12 }
```



La Regla de Oro: Si heredas de una clase abstracta, estás **OBLIGADO** a implementar sus métodos abstractos (darles código). Si no lo haces, tú también serás abstracto.

3.3. Ejemplo: Geometría

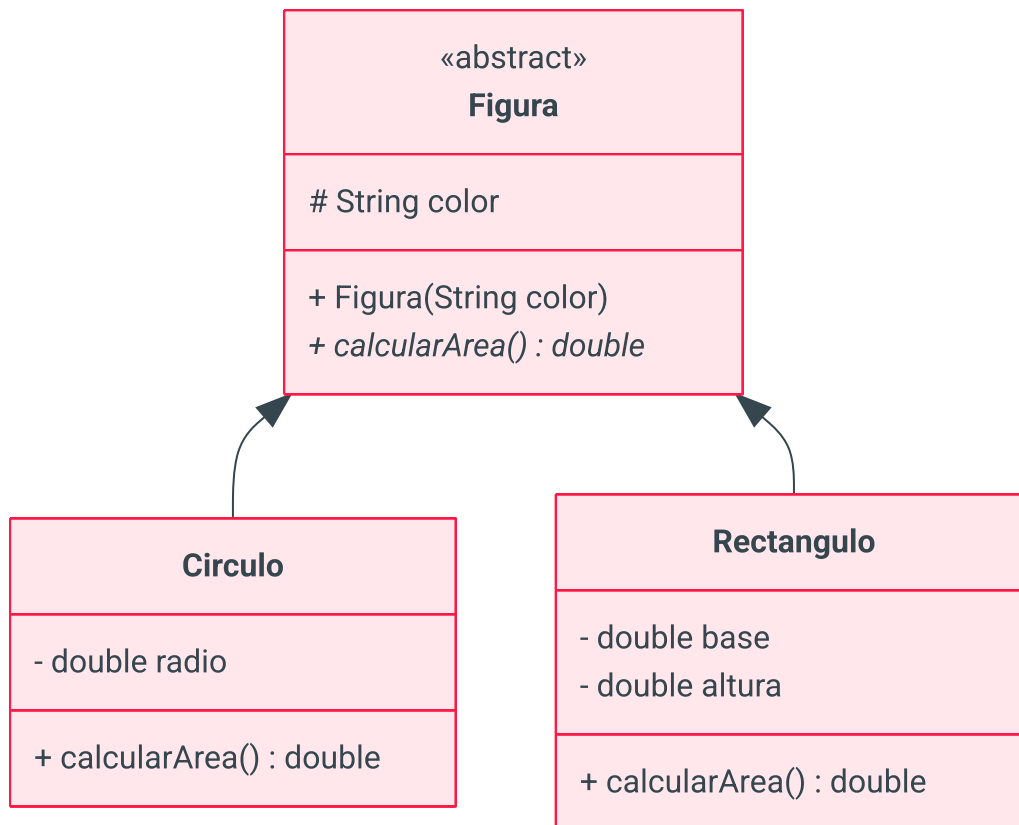
```
1 // CLASE HIJA 1
2 public class Circulo extends Figura {
3     private double radio;
4
5     public Circulo(String color, double radio) {
6         super(color);
7         this.radio = radio;
8     }
9
10    @Override
11    public double calcularArea() {
12        return Math.PI * radio * radio;
13    }
14 }
```

```
1 // CLASE HIJA 2
2 public class Rectangulo extends Figura {
3     private double base, altura;
4
5     public Rectangulo(String color, double base, double altura) {
6         super(color);
7         this.base = base;
8         this.altura = altura;
9     }
10
11    @Override
12    public double calcularArea() {
13        return base * altura;
14    }
15 }
```

```
1 // CLASE HIJA 3
2 public class Triangulo extends Figura {
3     private double base, altura;
4
5     public Triangulo(String color, double base, double altura) {
6         super(color);
7         this.base = base;
8         this.altura = altura;
9     }
10
11    @Override
12    public double calcularArea() {
13        return (base * altura) / 2;
14    }
15 }
```

3.4. Diagrama UML

En los diagramas de clases, las clases abstractas y los métodos abstractos se representan en *cursiva* (o con la etiqueta `<<abstract>>`).



(Nota: El asterisco `*` o la cursiva en `calcularArea()` indica que es abstracto).

3.5. Clases y Métodos Finales (`final`)

Justo lo opuesto a `abstract` es `final`.

- **Clase `final`**: "Estéril". Nadie puede heredar de ella. (Ej: `String` es `final` por seguridad).
- **Método `final`**: "Intocable". Nadie puede sobrecribirlo (`@Override`).

```

1 // Ejemplo: Una clase que no queremos que nadie modifique ni extienda
2 public final class ConstantesFisicas {
3     public static final double GRAVEDAD = 9.81;
4 }
5
6 // ERROR DE COMPILACIÓN: No se puede heredar de una clase final
7 // public class MiFisica extends ConstantesFisicas { ... }
  
```



Momento de Práctica: Gestión de Nóminas

Vamos a crear un sistema de pago para una empresa.

1. Clase Abstracta `Empleado` :

- Atributos: `nombre`, `dni`.
- Método Abstracto: `double calcularSueldo()`.
- Método Normal: `toString()` que devuelva el nombre.

2. Hija `EmpleadoAsalariado` :

- Tiene `sueldoBase` (fijo) y `antiguedad` (años).
- `Sueldo = sueldoBase + (100 * antiguedad)`.

3. Hija `Freelance` :

- Tiene `precioHora` y `horasTrabajadas`.
- `Sueldo = precioHora * horasTrabajadas`.

4. Main (Polimorfismo puro):

- Crea un `ArrayList<Empleado>`.
- Añade un Asalariado y un Freelance.
- Recorre la lista y suma cuánto dinero total tiene que pagar la empresa este mes.

4. Interfaces

Las **Interfaces** llevan la abstracción al siguiente nivel. Si una Clase Abstracta define **QUÉ ERES** (Identidad), una Interfaz define **QUÉ PUEDES HACER** (Capacidad).

4.1. Metáfora: El Mando de la Consola

Imagina el mando de una videoconsola.

- Tiene unos botones definidos: **X, O, Triángulo, Cuadrado**.
- Si juegas al **FIFA**, al pulsar **X** haces un *Pase*.
- Si juegas al **Call of Duty**, al pulsar **X** haces un *Salto*.

El mando (Interfaz) impone **qué botones existen**, pero no **qué hacen**. La clase concreta (el juego) es quien decide cómo reaccionar a esos botones.

4.2. Diferencias Clave



Característica	Clase Abstracta	Interfaz
Concepto	Identidad (ADN). "Soy un Animal".	Habilidad (Título). "Sé Nadar".
Relación	Padres/Hijos (Jerarquía Vertical).	Contrato (Conexión Horizontal).
Atributos	Puede tener variables normales.	Solo constantes (<code>public static final</code>).
Herencia	Simple (Solo un padre).	Múltiple (Puedes tener muchas habilidades).

4.3. Ejemplo: GPS Universal (`Localizable`)

Imagina una App de mapas. Necesita mostrar cosas en el mapa. No solo coches. También personas, mascotas con collar GPS, o incluso edificios. No tienen NADA en común (no son familia), pero todos son **Localizables**.

Primero, vamos a definir una clase que almacene las coordenadas.

```

1  public class Coordenadas {
2
3      public double latitud, longitud;
4
5      public Coordenadas(double latitud, double longitud) {
6          this.latitud = latitud;
7          this.longitud = longitud;
8      }
9  }
```

Ahora, vamos a definir la interfaz `Localizable`. Estará implementada por todas las clases que quieran ser localizables y usaremos el objeto `Coordenadas` para almacenar las coordenadas en cada clase.

```

1  // 1. Definimos la CAPACIDAD (Interfaz)
2  public interface Localizable {
3      // "Quien firme esto, debe saber decirme dónde está"
4      double getLatitud();
5      double getLongitud();
6  }
```

Podemos usar la interfaz `Localizable` para implementar la localización en clases abstractas.



```
1 public abstract class Vehiculo implements Localizable {
2
3     protected Coordenadas gps;
4
5     /** Simplificamos el código */
6
7     @Override
8     public double getLatitud() {
9         return this.gps.lat;
10    }
11
12    @Override
13    public double getLongitud() {
14        return this.gps.lon;
15    }
16
17 }
```

Podemos usar la interfaz `Localizable` para implementar la localización en clases concretas.

```
1 public class Usuario implements Localizable {
2
3     private String nombre;
4     private int edad;
5     private Coordenadas movil;
6
7     /** Simplificamos el código */
8
9     public Usuario(String nombre, int edad, Coordenadas movil) {
10         this.nombre = nombre;
11         this.edad = edad;
12         this.movil = movil;
13     }
14
15     @Override
16     public double getLatitud() {
17         return this.movil.lat;
18     }
19
20     @Override
21     public double getLongitud() {
22         return this.movil.lon;
23     }
24
25 }
```

```
1 public class Perro implements Localizable {
2
3     private String nombre;
4     private Coordenadas collar;
5
6     /** Simplificamos el código */
7
8     public Perro(String nombre, Coordenadas collar) {
9         this.nombre = nombre;
```

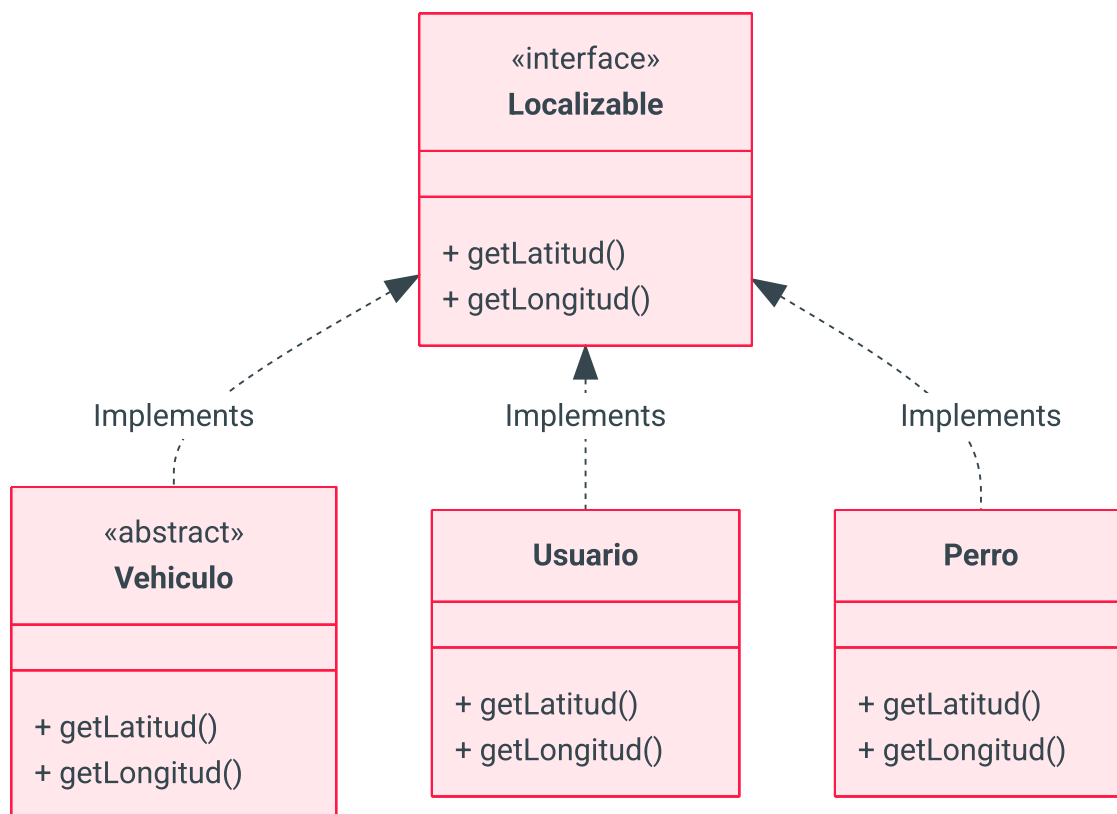
```
10         this.collar = collar;
11     }
12
13     @Override
14     public double getLatitud() {
15         return this.collar.lat;
16     }
17
18     @Override
19     public double getLongitud() {
20         return this.collar.lon;
21     }
22
23 }
```

Aquí tenemos un ejemplo de uso en el `main`:

```
1  public class MainInterfaces {
2
3      public static void main(String[] args) {
4
5          Coordenadas localizacion = new Coordenadas(39.67, -0.25);
6
7          Perro perro = new Perro("Sparky", localizacion);
8          Usuario usuario = new Usuario("Dulcinea del Toboso", 18,
9 localizacion);
10
11          System.out.println("Mi perro está en " + perro.getLatitud() + " "
12 + perro.getLongitud());
13          System.out.println("Dulcinea del Toboso está en " +
14 usuario.getLatitud() + " " + usuario.getLongitud());
15
16      }
17 }
```

4.4. Diagrama UML

En UML, la relación **"implementa una interfaz"** (Realización) se dibuja con una **línea discontinua y punta triangular hueca**.



Poder Ilimitado: Implementar Múltiples Interfaces

A diferencia de la herencia de clases (`extends`), donde solo puedes tener **un padre**, en Java puedes implementar **tantas interfaces como quieras**.

Solo tienes que separarlas por comas: `public class SuperHeros implements Volador, SuperFuerza, IdentidadSecreta { ... }`

¡Eso sí! Tienes que escribir el código de **TODOS** los métodos de **TODAS** las interfaces.



Momento de Práctica: La Casa Domótica

Vamos a crear los dispositivos inteligentes de una casa.

1. **La Interfaz:** Crea `Controlable` con métodos `void encender()` y `void apagar()`.
2. **Las Clases:** Crea `Televisor`, `AireAcondicionado` y `Persiana` que implementen `Controlable`.
 - *Piensa* en qué atributo necesitas para indicar si un dispositivo está encendido o apagado.
 - *Detalle:* El televisor al encenderse dice: "Bienvenido a Netflix".
 - *Detalle:* El aire al apagarse dice: "Cerrando aspas...".
3. **La Prueba:** En el `main`, crea un objeto de cada uno. Enciéndelos y apágalos.
4. **Reflexión:** Fíjate que aunque los métodos se llaman igual, cada objeto hace algo distinto (uno muestra logos, otro mueve aspas). Eso es gracias a la interfaz.

5. Polimorfismo

El **Polimorfismo** ("muchas formas") es la magia real de la POO. Nos permite tratar a objetos diferentes de forma genérica, pero conservando su comportamiento único.

5.1. Metáfora: La Caja Mágica

Imagina que tienes una caja etiquetada como "**Personaje**".

- Dentro de la caja metes a un **Guerrero**.
- Tú, desde fuera, solo ves la etiqueta "Personaje".
- Si le gritas a la caja "¡ATACA!", lo que saldrá de dentro es un **ESPADAZO** (comportamiento de Guerrero), aunque tú trataras la caja como algo genérico.

Concepto Clave:

- **Tipo de la Variable (La Caja):** Define qué métodos PUEDES llamar.
- **Tipo del Objeto (El Contenido):** Define CÓMO se ejecutan esos métodos.

5.2. Ligadura Dinámica (Dynamic Binding)

```
1 // Creamos un ARRAYLIST de la clase padre (La Caja)
2 ArrayList<Personaje> grupo = new ArrayList<>();
3
4 // Guardamos objetos HIJOS (El Contenido)
```

```
5 grupo.add(new Guerrero("Conan", 100, 5, 20));
6 grupo.add(new Mago("Gandalf", 80, 10, 50));
7
8 // Creamos un enemigo dummy
9 Personaje enemigo = new Orco("Orokhur", 200, 1, 8);
10
11 // BUCLE POLIMÓRFICO:
12 // Tratamos a todos como "Personaje", pero cada uno reacciona a su
13 manera.
14 for (Personaje p : grupo) {
15     p.atacar(enemigo); // JAVA VE: Guerrero -> Espadazo, Mago -> Fuego...
16 }
```

Java decide **en tiempo de ejecución** qué código ejecutar. Esto se llama **Ligadura Dinámica**.

5.3. Trasteando con la Herencia: Casting y instanceof

A veces, tienes un `Personaje` (caja genérica) pero sabes que dentro hay un `Guerrero` y quieres usar su método especial `gritoDeGuerra()` (que el Mago no tiene).

Intentarlo directamente da error:

```
1 Personaje p = grupo.get(0); // Recuperamos al primero (Sabemos que es
2 Conan)
   // p.gritoDeGuerra(); // ERROR: La clase Personaje no tiene ese método.
```

Para solucionarlo, necesitamos hacer **Casting** (destapar la caja y confirmar qué es). Pero ¡cuidado! Si te equivocas, el programa explota. Para estar seguros, usamos el "escáner de rayos X": `instanceof`.

```
1 if (p instanceof Guerrero) {
2     // Confirmado: Es un guerrero. Hacemos Downcasting seguro.
3     Guerrero g = (Guerrero) p;
4     g.gritoDeGuerra(); // ¡AHORA SÍ!
5 }
```



📖 Reto Final de Unidad: El Zoológico Interactivo 🐱🐶🐙

Vamos a crear un simulador de zoológico con gestión de energía.

1. Clase Abstracta `Animal` :

- Atributos: `nombre`, `energia` (empieza en 50).
- Método `jugar(int gasto)` : Resta energía e imprime "[Nombre] está jugando... Energía restante: [X]".
- Método Abstracto `comer(String comida)` : Cada animal recupera energía de forma distinta según lo que coma. Si no quiere la comida, muestra rechazo con un mensaje.

2. Interfaz `Trucos` :

- Método `hacerTruco()` : Muestra un mensaje especial.

3. Las Especies:

- **Leon** : Solo come "Carne" (+100 de energía). Si le das otra cosa, se enfada y pierde 10. *No hace trucos.*
- **Delfin** : Come "Peces" (+50) y "Calamar" (+80). Implementa `Trucos` : "¡Hace un salto mortal hacia atrás!" y resta 10 de energía.
- **Perro** : Come de todo ("Carne" +50, "Pienso" +30). Implementa `Trucos` : "¡Da la patita y rueda!" y resta 5 de energía.

4. El Show (Main):

- Crea un `ArrayList<Animal>` con uno de cada.
- **Bucle del Show:**
 - a. Haz que todos **jueguen** (gastan 20 de energía).
 - b. Aliméntalos a todos con "Carne". (Observa cómo el Delfín no la querrá).
 - c. Si el animal sabe hacer **trucos** (`instanceof`), ¡que actúe para el público!

5.4. Sobreescritura de Comparadores (Igualdad y Orden)

En Java, a menudo necesitamos saber si dos objetos son "iguales" o cuál va antes en una lista (ordenar). Por defecto, `==` solo compara si son **el mismo objeto en memoria**, no si tienen los mismos datos.

5.4.1. Igualdad de Contenido (`equals`)

Para comparar el **contenido**, debemos sobreescibir el método `equals` y `hashCode`.

```
1 @Override
2 public boolean equals(Object obj) {
3     if (this == obj) return true;
```

```

4      if (obj == null || getClass() != obj.getClass()) return false;
5
6      Personaje otro = (Personaje) obj;
7      // Decidimos que dos personajes son iguales si tienen el mismo nombre
8      return this.nombre.equals(otro.nombre);
9  }
10
11  @Override
12  public int hashCode() {
13      // Si dos objetos son iguales, DEBEN tener el mismo hashCode.
14      return this.nombre.hashCode();
15  }

```



El contrato de `equals` y `hashCode`

1. `equals()` : Define cuándo dos objetos son lógicamente "el mismo".
2. `hashCode()` : Devuelve un número entero que representa al objeto (como una huella dactilar rápida).

Regla de Oro: Siempre que sobrescribas `equals`, **DEBES** sobrescribir `hashCode`. ¿Por qué? Porque colecciones como `HashSet` o `HashMap` usan primero el `hashCode` para buscar rápido. Si dos objetos son iguales pero tienen `hashCode` distinto, estas colecciones fallarán y crearán que son objetos diferentes.

5.4.2. Orden Natural (`Comparable`)

Si queremos poder ordenar una lista de personajes con `Collections.sort(lista)`, nuestra clase debe implementar la interfaz `Comparable<T>` y su método `compareTo`.

Ejemplo 1: Ordenar por Nombre (Texto)

```

1  public class Personaje implements Comparable<Personaje> {
2      // ... codigo ...
3
4      @Override
5      public int compareTo(Personaje o) {
6          // Orden alfabético por nombre (String ya implementa compareTo)
7          return this.nombre.compareTo(o.nombre);
8      }
9  }

```

Ejemplo de Uso:

```

1  ArrayList<Personaje> lista = new ArrayList<>();
2  lista.add(new Guerrero("Link", ...));
3  lista.add(new Mago("Gandalf", ...));
4  lista.add(new Guerrero("Aragorn", ...));
5
6  // Al llamar a sort, Java usa tu método compareTo para ordenar
7  Collections.sort(lista);

```



```

8
9   for (Personaje p : lista) {
10       System.out.println(p.getNombre()); // Salida: Aragorn, Gandalf, Link
11   }

```

Ejemplo 2: Ordenar por Nivel (Numérico)

Si preferimos ordenar por su poder (nivel), comparamos enteros.

```

1   @Override
2   public int compareTo(Personaje o) {
3       // Orden numérico por nivel
4       return Integer.compare(this.nivel, o.nivel);
5   }

```

!!! tip "Comparadores de tipos `Integer` y `Double` Java ya tiene implementados los comparadores para estos tipos.

```

1   *   `Integer.compare(a, b)`
2   *   `Double.compare(a, b)`

```

6. Ejemplo Completo de Integración

A continuación, tienes un ejemplo de código completo que integra **Herencia, Clases Abstractas, Interfaces y Polimorfismo**. Puedes copiar cada bloque en un archivo separado para probarlo.

Escenario: Un pequeño juego RPG.

6.1. La Interfaz (Capacidad)

```

1   // Archivo: Atacable.java
2   public interface Atacable {
3       void recibirDano(int cantidad);
4   }

```

6.2. La Clase Abstracta (Identidad Base)

```

1   // Archivo: Personaje.java
2   public abstract class Personaje implements Atacable,
3   Comparable<Personaje> {
4
5       protected String nombre;
6       protected int vida;
7       protected int nivel;
8

```

```

9      public Personaje(String nombre, int vida, int nivel) {
10          this.nombre = nombre;
11          this.vida = vida;
12          this.nivel = nivel;
13      }
14
15      public abstract void atacar(Personaje objetivo);
16
17      @Override
18      public void recibirDano(int cantidad) {
19          this.vida -= cantidad;
20          if (this.vida < 0) this.vida = 0;
21          System.out.println(this.nombre + " recibe " + cantidad + " de
22 daño. Vida restante: " + this.vida);
23      }
24
25      public boolean estaVivo() {
26          return this.vida > 0;
27      }
28
29      public String getNombre() { return nombre; }
30
31      // --- MÉTODOS DE IGUALDAD Y ORDEN ---
32
33      @Override
34      public boolean equals(Object obj) {
35          if (this == obj) return true;
36          if (obj == null || getClass() != obj.getClass()) return false;
37          Personaje other = (Personaje) obj;
38          return this.nombre.equals(other.nombre);
39      }
40
41      @Override
42      public int hashCode() {
43          return this.nombre.hashCode();
44      }
45
46      @Override
47      public int compareTo(Personaje o) {
48          // Orden alfabético por nombre
49          return this.nombre.compareTo(o.nombre);
50      }
51  }

```

6.3. Las Clases Concretas (Implementación)

```

1  // Archivo: Guerrero.java
2  public class Guerrero extends Personaje {
3      private int furia;
4
5      public Guerrero(String nombre, int vida, int nivel, int furia) {
6          super(nombre, vida, nivel);
7          this.furia = furia;
8      }
9  }

```



```

10     @Override
11     public void atacar(Personaje objetivo) {
12         int dano = this.nivel * 2 + this.furia;
13         System.out.println(this.nombre + " lanza un ESPADAZO BRUTAL a " +
14 objetivo.getNombre());
15         objetivo.recibirDano(dano);
16     }
17
18     public void gritoDeGuerra() {
19         System.out.println(this.nombre + ": ¡¡GRRRR!!");
20     }
21 }

```

```

1 // Archivo: Mago.java
2 public class Mago extends Personaje {
3
4     private int mana;
5
6     public Mago(String nombre, int vida, int nivel, int mana) {
7         super(nombre, vida, nivel);
8         this.mana = mana;
9     }
10
11     @Override
12     public void atacar(Personaje objetivo) {
13         if (this.mana >= 10) {
14             int dano = this.nivel * 3; // El mago pega más fuerte
15             this.mana -= 10;
16             System.out.println(this.nombre + " lanza BOLA DE FUEGO a " +
17 objetivo.getNombre());
18             objetivo.recibirDano(dano);
19         } else {
20             System.out.println(this.nombre + " no tiene maná y golpea con
21 el bastón (puf...)");
22             objetivo.recibirDano(1);
23         }
24     }
25 }

```

```

1 // Archivo: Orco.java
2 public class Orco extends Personaje {
3
4     private int rabia;
5
6     public Orco(String nombre, int vida, int nivel, int rabia) {
7         super(nombre, vida, nivel);
8         this.rabia = rabia;
9     }
10
11     @Override
12     public void atacar(Personaje objetivo) {
13         if (this.vida < 10) {
14             int dano = this.nivel * 4 + rabia;
15             System.out.println(this.nombre + " ataca con FURIA " +
16 objetivo.getNombre());

```

```

17         objetivo.recibirDano(dano);
18     } else {
19         objetivo.recibirDano(this.nivel);
20     }
21 }

```

6.4. El Main (Polimorfismo en Acción)

```

1  // Archivo: JuegoRPG.java
2  import java.util.ArrayList;
3
4  public class JuegoRPG {
5      public static void main(String[] args) {
6          ArrayList<Personaje> equipo = new ArrayList<>();
7
8          // Polimorfismo: Guardamos Hijos en variables Padre (lista)
9          equipo.add(new Guerrero("Conan", 150, 5, 100));
10         equipo.add(new Mago("Gandalf", 80, 10, 200));
11
12         // Enemigo dummy para probar
13         Guerrero enemigo = new Orco("Orco Feo", 200, 3, 8);
14
15         System.out.println("--- EMPIEZA LA BATALLA ---");
16
17         // Bucle Polimórfico
18         for (Personaje p : equipo) {
19             p.atacar(enemigo); // Java decide qué atacar() usar
20             dinámicamente
21
22             // Uso de instanceof para acceder a métodos exclusivos
23             if (p instanceof Guerrero) {
24                 ((Guerrero) p).gritoDeGuerra(); // Downcasting seguro
25             }
26         }
27     }
}

```

6.5. Diagrama UML Completo del Juego de Rol

